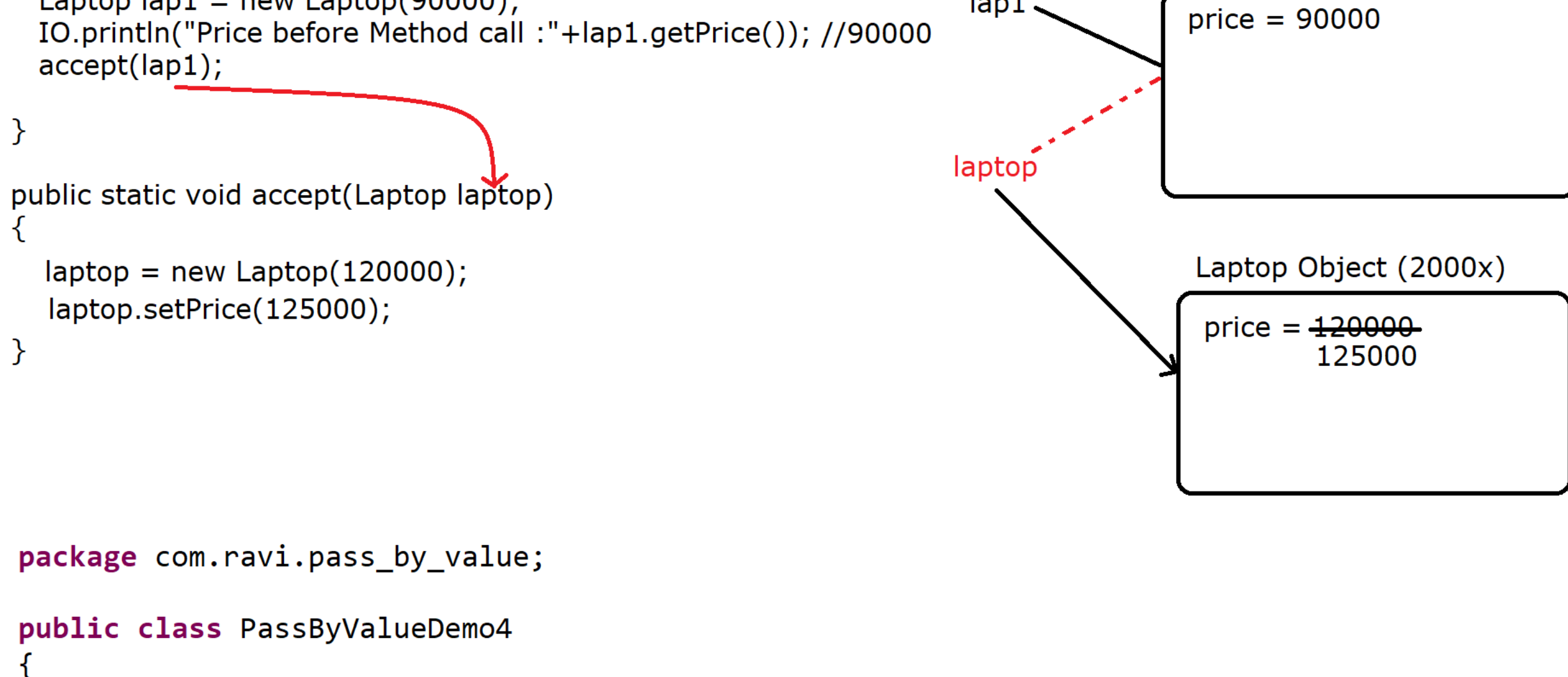


pass by Value with two different objects :

Diagram for PassByValueDemo4.java



```
package com.ravi.pass_by_value;

public class PassByValueDemo4
{
    public static void main(String[] args)
    {
        Laptop lap1 = new Laptop(90000);
        IO.println("Price before Method call :"+lap1.getPrice()); //90000
        accept(lap1);
        IO.println("Price After Method call :"+lap1.getPrice()); //90000
    }

    public static void accept(Laptop laptop)
    {
        laptop = new Laptop(120000);
        laptop.setPrice(125000);
        IO.println("New Object Price :"+laptop.getPrice()); //125000
    }
}

class Laptop
{
    private double price;

    public Laptop(double price)
    {
        super();
        this.price = price;
    }

    public void setPrice(double price)
    {
        this.price = price;
    }

    public double getPrice()
    {
        return this.price;
    }
}
```

Working with different types of return type of the method :

* As we know, In java without a valid return type, we cannot declare a method in Java that means return type is compulsory to define of a method.

* As a method return type we have various options are available :

- void as a return type [Method will not return any value]
- All primitive data types like byte, short, int, long, float, double, char and boolean
- We can use **reference type** also as a method return type like method return type can be a class, interface, array and so on

Case 1:

public void accept()
{
 return; //compiler will add to return the control to the caller method
}

Case 2:

public int accept()
{
 return 10;
}

Case 3:

public int accept()
{
 return 10.1; //error (The return value i.e. 10.1 is not compitble with return type of the method i.e. int type)
}

Note : Conclusion is, the **return value** of a method must be compitble with **return type** of the method

Case 4:

public class Test
{
 public Test accept()
 {
 Test t1 = new Test(); * The object of Test class i.e new Test() we can return here
 return t1; * Object of Test i.e. new Test(), we can assign to return type of the method i.e Test
 }
}

Case 5 :

package com.ravi.method_return_type;

public class Test
{
 public Test(int x)
 {
 }

 public Test accept()
 {
 Test t1 = new Test(9);
 return t1;
 }
}

Note : Method return type is a class so, we will depend upon constructor to create the object then only we can return the object

What is static factory Method ?

A static factory method is special **static method** which is used to **create and return** the object of the class instead of using constructor directly.

It prevents us to create the object from outside of the BLC class that means outsider cannot create the object because constructor is private [Singleton design pattern]

[Note: It is not Factory design pattern, Factory design pattern we can achieve using **interface** concept]

```
package com.ravi.static_factory_method;

public class BookDemo
{
    public static void main(String[] args)
    {
        String title = IO.readLine("Enter Book Title :");
        String author = IO.readLine("Enter Book Author :");

        Book book = Book.getBookObject(title, author);
        IO.println(book);
    }
}

class Book
{
    private String title;
    private String author;

    private Book(String title, String author)
    {
        super();
        this.title = title;
        this.author = author;
    }

    @Override
    public String toString()
    {
        return "Book [title=" + title + ", author=" + author + "];"
    }

    //static factory method
    public static Book getBookObject(String title, String author)
    {
        Book b1 = new Book(title, author);
        return b1;
    }
}
```